

SchedTwin: Simulation-Assisted Online Learning for Adaptive HPC Scheduling

Yihe (Jordan) Zhang, Kanglin Xu, Yash Kurkure, Michael E. Papka, Zhiling Lan

Motivation

- Modern HPC workloads are highly **dynamic and heterogeneous**, making static heuristic schedulers suboptimal.
- Existing approaches face a trade-off: heuristics **lack adaptability**, while DRL **incurs high training and retraining cost**. Crucially, no single scheduling policy performs well across all system states.
- This motivates **learning to adaptively select the right policy at the right time**, rather than designing a single optimal policy.

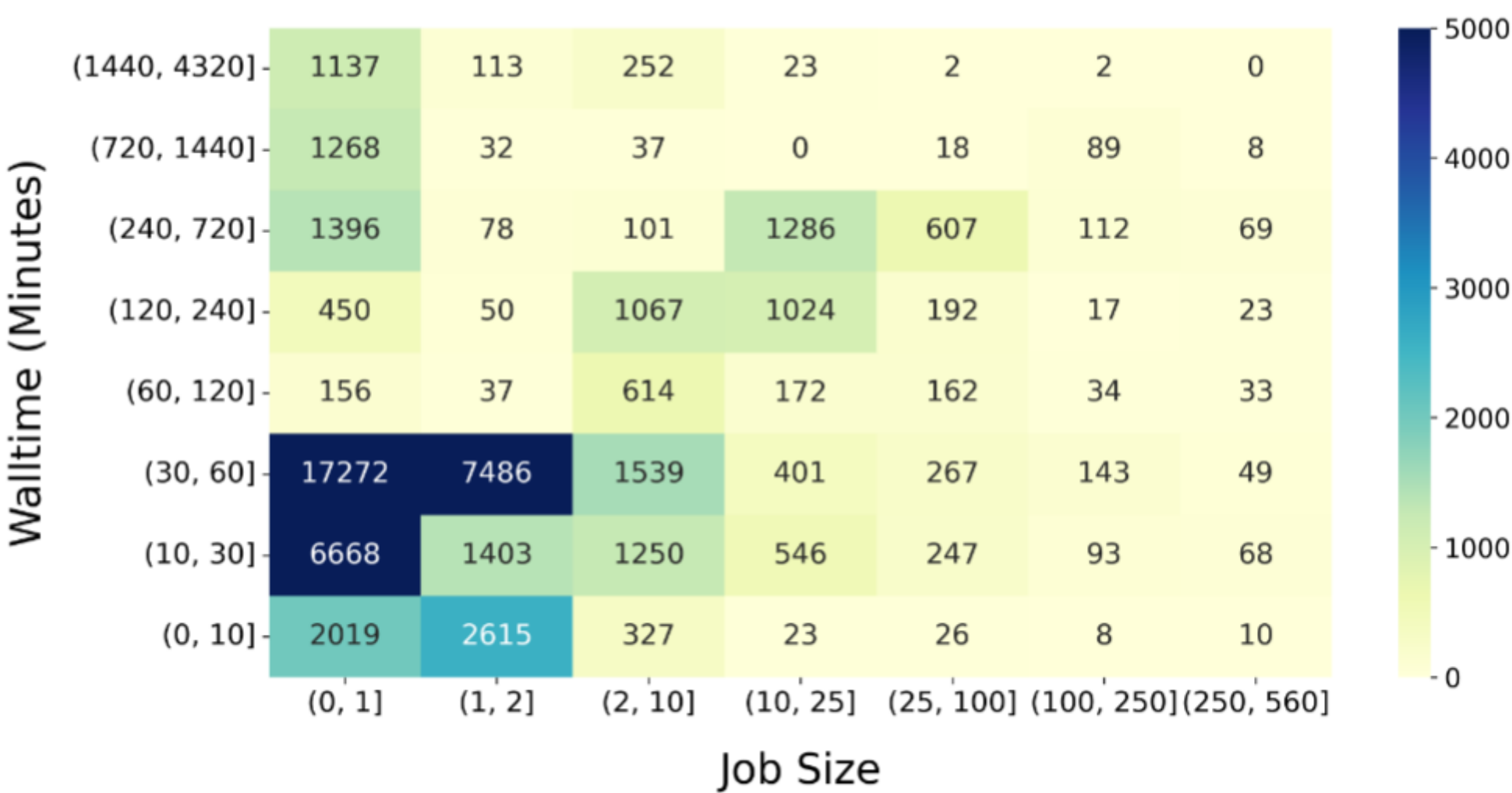


Figure 1. Job distribution on the ALCF Polaris system (April – May 2024), showing the significant variance in job sizes and requested walltimes.

Experiment Setup

Workloads and Systems:

Real-world production traces from **ALCF supercomputers**:

- Polaris** (560 nodes)
- Theta** (4,392 nodes)
- Aurora** (10,624 nodes)

Evaluation Environments:

Trace-driven Emulation (extended from CQSim):
Replays real scheduling decisions offline

Hardware:

Intex Xeon CPUs

Baselines: FCFS w. Backfilling, WFP, F1, UNI, SJF, LJF, RLScheduler

System Design

Core Idea

SchedTwin treats scheduling as a **policy selection problem** and combines:

Online learning (CMAB) → learn which policy works best

Fast simulation → predict future impact before decision

Enables **adaptive, look-ahead scheduling** without training.

System Architecture (Closes-loop design)

Event Stream (Real-time State)

Collects job and resource events

Builds system context x_t^{sys}

Non-intrusive integration with production schedulers

Fast Scheduling Simulator (Predictive Signal)

Runs *what-if simulations* for each candidate policy

Estimates metrics (wait time, slowdown, etc.)

Produces predictive context $x_{t,a}^{pred}$

Contextual Multi-Armed Bandit (CMAB Agent)

Input: combined context $[x_t^{sys}, x_{t,a}^{pred}]$

Output: best policy at current timestep

Balances **exploration vs. exploitation**

Learns online without retraining

Policy Feedback Loop

Apply selected policy to real scheduler

Collect delayed reward (multi-cycle impact)

Update bandit model continuously

SchedTwin integrates **simulation plus online bandit learning** to make context-aware, forward-looking scheduling decisions in real time.

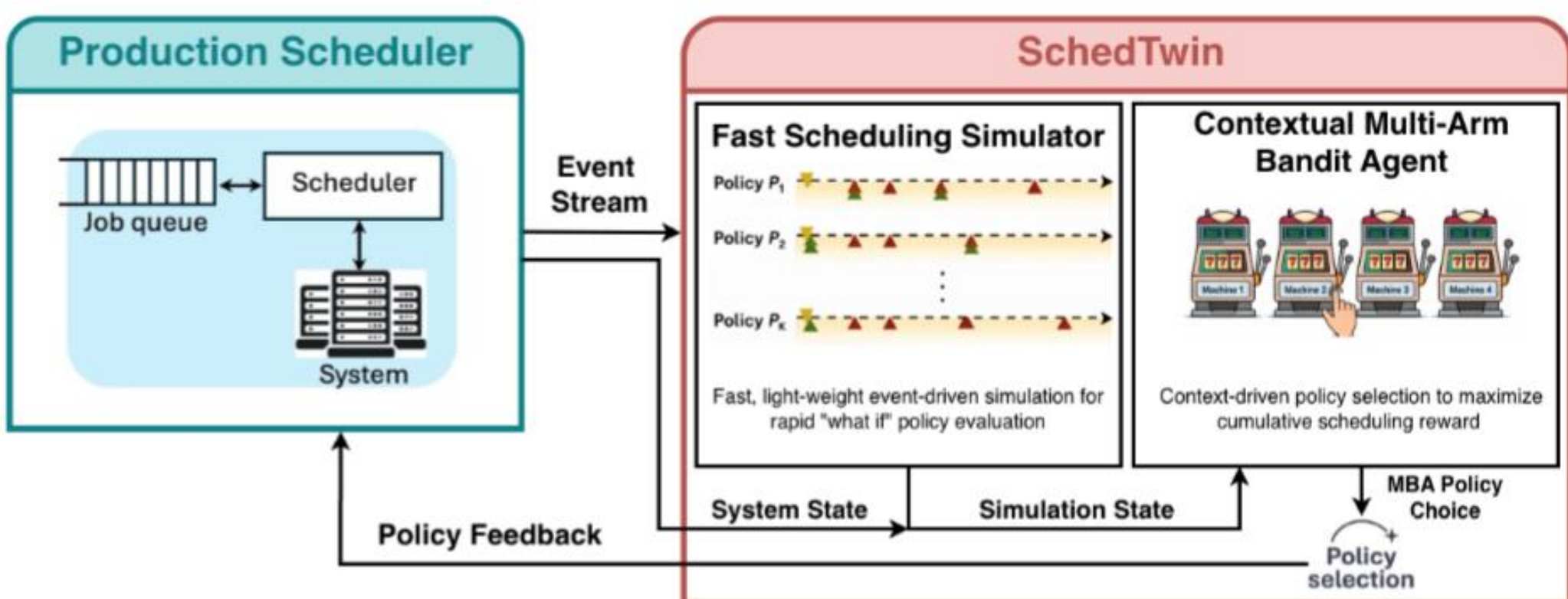


Figure 2. Overview of *SchedTwin*, a simulation-assisted contextual bandit framework for adaptive HPC scheduling. *SchedTwin* (in red) observes the production scheduler, evaluates candidate scheduling policies via the simulation-assisted bandit agent, and adaptively selects the policy that maximizes long-term scheduling performance.

Scheduling Performance

Policy	Wait time (s)		
	GeoMean	Mean	P98
FCFS	13.21	5551.02	51459.80
SJF	18.12	47465.44	1010820.80
LJF	175441.37	1488607.45	3350477.40
UNI	6.13	4312.69	40062.80
F1	3.53	3656.89	25377.80
RLScheduler	5965.84	62752.86	408000.06
WFP	9.59	4116.84	34839.00
SchedTwin	3.56 (62.9%↑)	3011.68 (26.8%↑)	20830.60 (40.2%↑)

	Slowdown			System Utilization
	GeoMean	Avg	P98	
	2.41	87.17	708.80	0.786
	2.95	743.40	1608.50	0.749
	1334.84	47970.59	303003.32	0.738
	1.71	58.78	247.44	0.783
	1.46	45.08	136.08	0.776
	38.51	5939.46	13061.82	0.748
	2.03	59.65	207.90	0.788
	1.45 (28.6%↑)	35.50 (40.5%↑)	114.67 (44.8%↑)	0.786 (0.25%↓)

Overhead

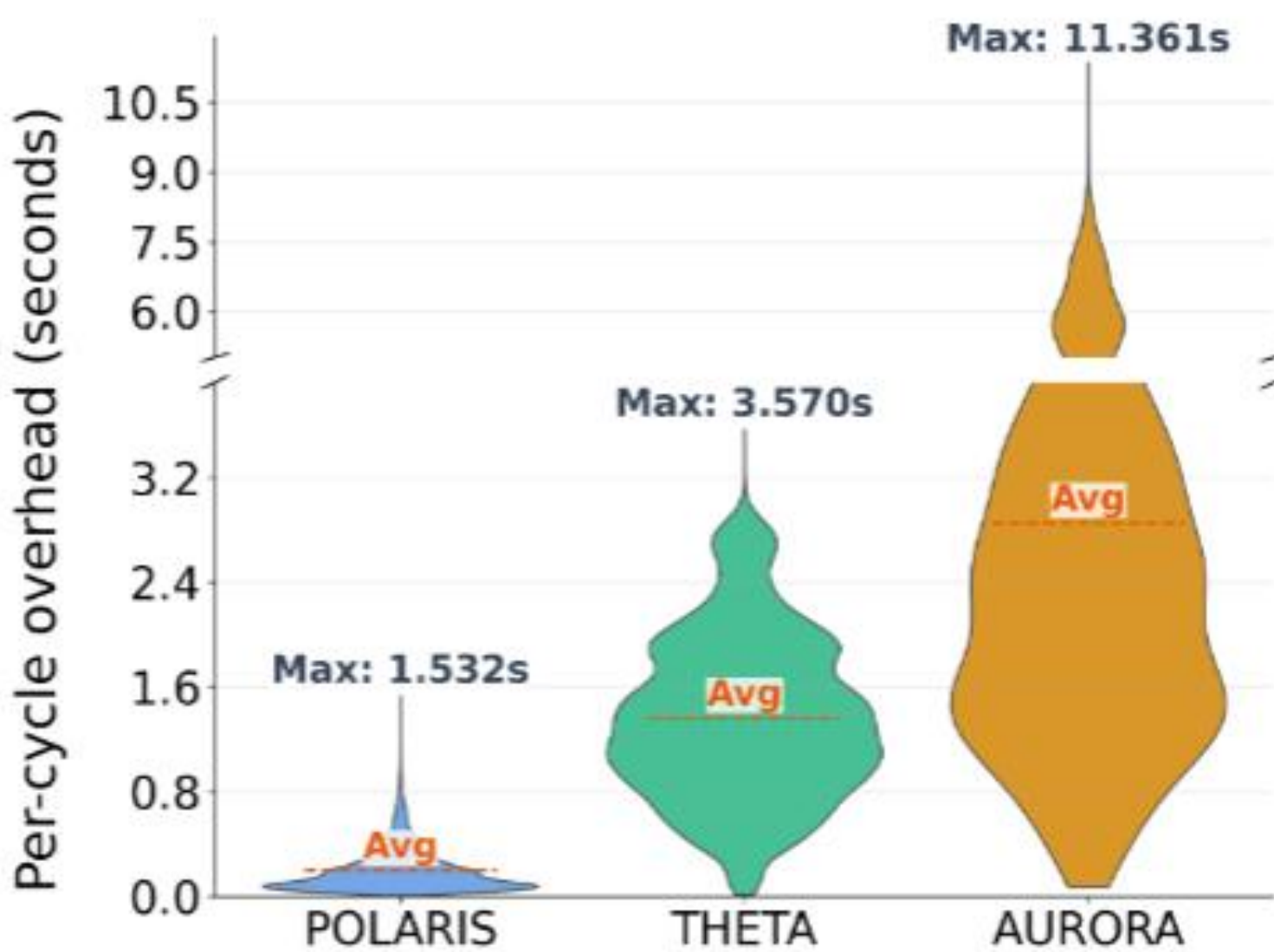


Table 1. (Left) Scheduling performance on **Polaris**. Lower is better for all metrics except system utilization, where higher is better. The deployed policy WFP is highlighted in **Blue**; SchedTwin is highlighted in **Red**.

Figure 3. (Right) Per-cycle runtime overhead of SchedTwin across three production systems, shown as violin plots with average values marked (dashed line).